

Instructions

Instruction execution - recap

- Program instructions reside in memory
- The PSW contains the address of the next instruction in the Instruction Address (IA) field
- The CPU picks an instruction for execution from the IA field, and loads the next instruction in IA
- If there is a Branch (or Jump), the branch target is loaded on the IA (and so becomes the next instruction to be executed)
- Next: what do instructions look like?

Instruction formats

- Instructions are 1, 2, or 3 halfwords in length, and always start on a halfword boundary (an even address)
- Instructions start with an Operation Code (Opcode) followed by operands
- Operands can be registers, memory addresses, or “immediate” (the operand value itself)
- Instruction formats, presented next, are classified based on operand types

Instruction formats

- The original System/360 architecture had 5 instruction formats. They are still in use:

Format	Description
Register-and-register (RR)	Both operands are registers
Register-and-storage (RS)	One operand is a register
Register-and-indexed-storage (RX)	and another a storage address
Storage-and-immediate (SI)	One operand is a storage address and another an “immediate” value
Storage-and-storage (SS)	Both operands are storage addresses

- We’ll be referring to the formats using the abbreviations (RR, RS...)

Instruction formats

- As the architecture evolved, new instruction formats appeared
- There are now more than 30! Mostly, variations of the original five
- In the original formats, memory addresses are 12-bit base-displacement
- We will see just two more formats to show the use of long displacement and relative addressing
- We will introduce other formats as needed

Instruction formats

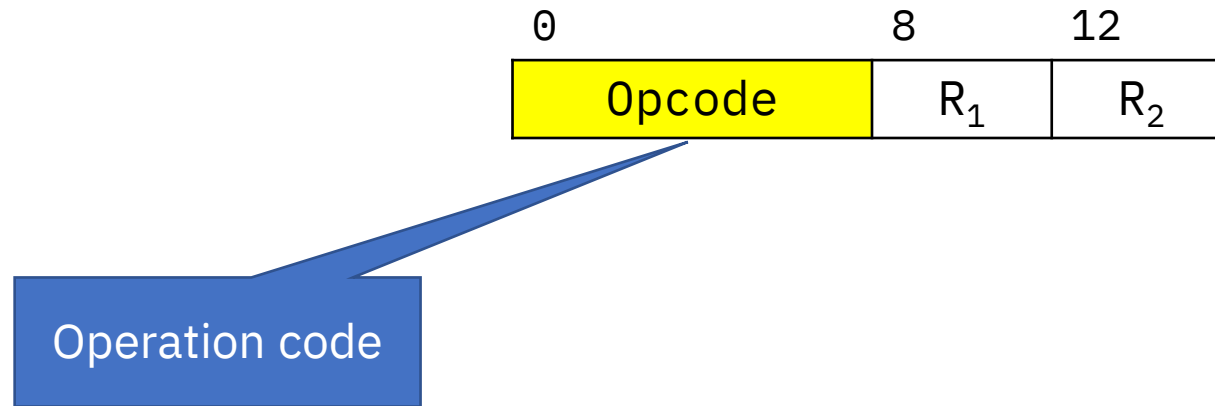
- You learned about long displacement and relative addressing earlier. Here are two formats that use them:

Format name	Description
Register-and-storage (RXY)	One operand is a register and another a long displacement address
Register-and-immediate (RI)	One operand is a register and another a 16-bit relative address

- We will now see each of the seven formats in more detail, with examples of real instructions, starting with RR

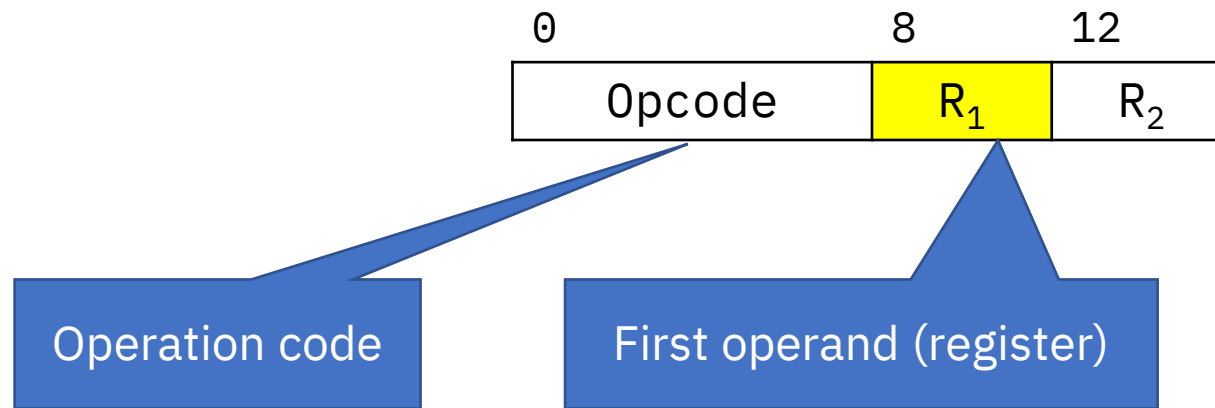
RR instructions

- RR instructions occupy 1 halfword
- In memory, they look like this:



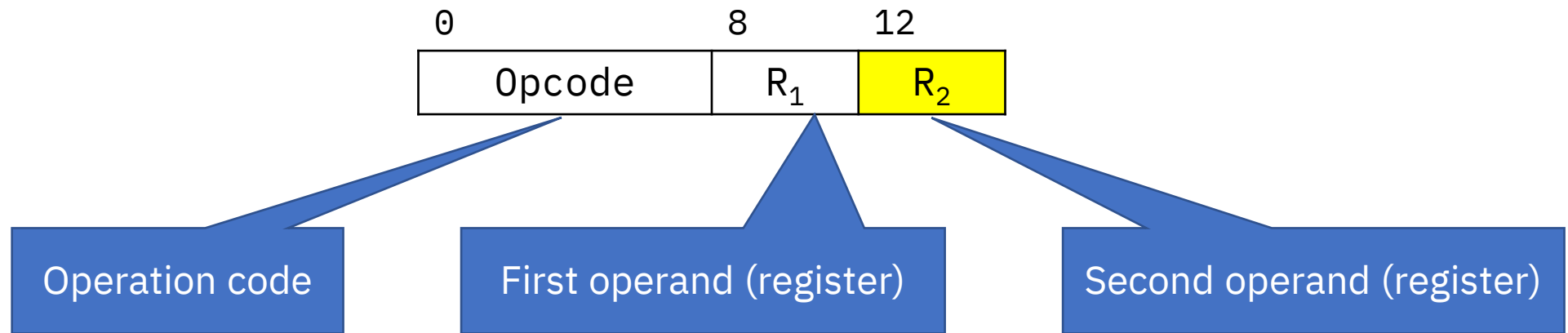
RR instructions

- RR instructions occupy 1 halfword
- In memory, they look like this:



RR instructions

- RR instructions occupy 1 halfword
- In memory, they look like this:



RR instructions - example

- Our first example of an RR instruction is ADD REGISTER
 - It adds two registers, and puts the result in the first register
- In Assembler language, each opcode has a mnemonic
 - The mnemonic for ADD REGISTER is **AR**
 - To code an instruction, you use the mnemonic
- Next, we'll see how you would code the instruction
“ADD REGISTER 14 to REGISTER 2”

RR instructions - example

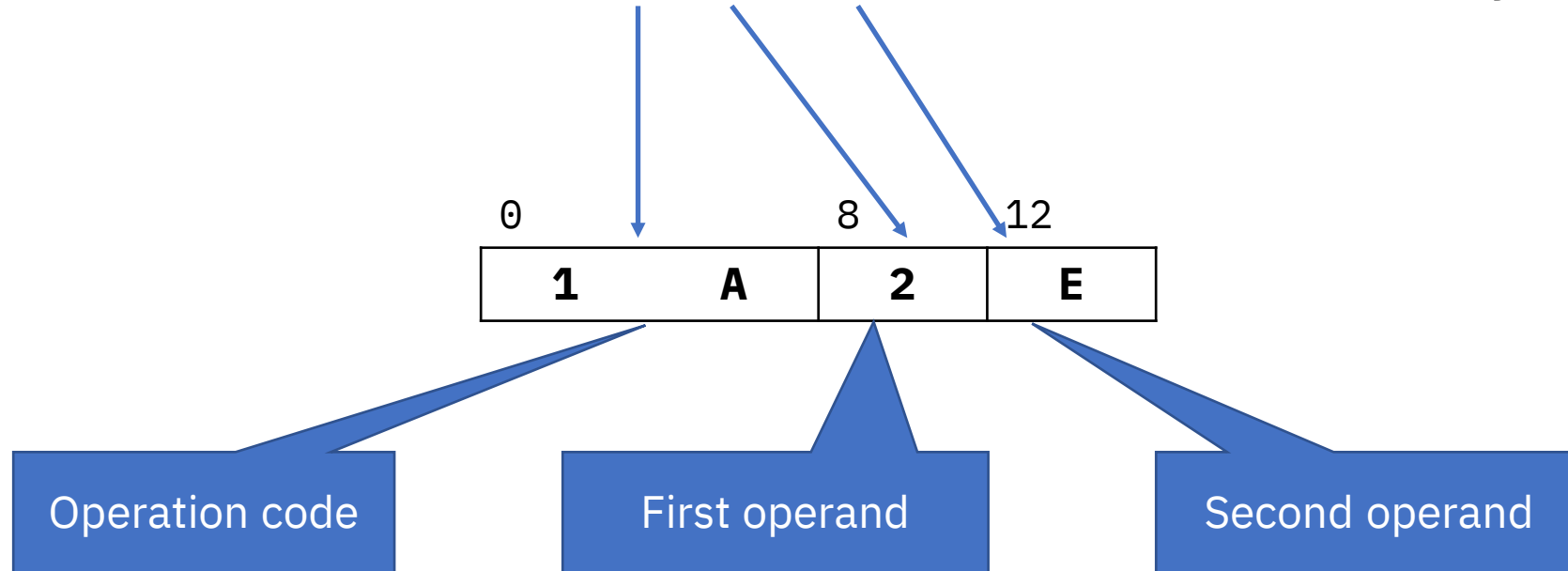
- This is the assembler statement for the ADD REGISTER instruction that adds register 14 to register 2 (your first encounter with real Assembler code!):

AR 2,14

- Throughout this section, we'll introduce a few Assembler concepts
 - Don't worry about the precise syntax for now, we'll cover that soon
- Note, in almost all z/Architecture instructions, *data moves “right to left”*: *from the second operand to the first*

RR instructions - example

- The opcode for ADD REGISTER is **1A**
- Our source statement (**AR 2, 14**) becomes, in memory:



RR instructions - another example

- Another RR instruction is SUBTRACT REGISTER
 - Mnemonic **SR**
 - Opcode **1B**
- This statement (“Subtract R12 from R8”)

SR 8,12

- ...becomes, in memory:

0	8	12
1	B	8
		C

Instruction formats

- Next, RS instructions

Format	Description
Register-and-register (RR)	Both operands are registers
Register-and-storage (RS)	One operand is a register
Register-and-indexed-storage (RX)	and another a storage address
Storage-and-immediate (SI)	One operand is a storage address and another an “immediate” value
Storage-and-storage (SS)	Both operands are storage addresses
Register-and-storage (RXY)	One operand is a register and another a long displacement address
Register-and-immediate (RI)	One operand is a register and another a 16-bit relative address